

Realize uma pesquisa sobre o tema de Privacidade e Segurança, ela deve ser dividida em: introdução, itens e subitens do tema, conclusão e referências. Nela devem constar a abordagem dos tópicos: Autenticação OAuth, JWT e segurança contra XSS/CSRF. Aprofunde-se nos temas porém considere que a totalidade do texto deve ter entorno de quatro páginas.

Pesquisa Sobre Privacidade e Segurança na Web

Privacidade e segurança em aplicações web dependem de mecanismos de **autenticação**, **autorização** e defesa contra ataques de navegador, sobretudo quando OAuth 2.0, JWT e proteções contra XSS/CSRF são combinados de forma correta. A literatura converge em um ponto: esses mecanismos são amplamente adotados e eficazes, mas sua segurança real depende menos do protocolo em si e mais de decisões de implementação, armazenamento de tokens, validação de redirecionamento e mitigação de falhas no cliente e no servidor (Hardt, 2012; Singh & Chaudhary, 2022; Pratama et al., 2026; Kshetri et al., 2024).

Introdução

No contexto de privacidade e segurança digital, aplicações web modernas precisam verificar identidades, limitar privilégios e proteger dados sensíveis sem expor credenciais dos usuários a terceiros (Vasylenko, 2025; Hardt, 2012; Pyroh et al., 2025). Esse cenário favoreceu a adoção de protocolos de autorização como o OAuth 2.0 e de tokens portáteis como JWT, ao mesmo tempo em que ampliou a relevância de ataques como Cross-Site Scripting (XSS) e Cross-Site Request Forgery (CSRF), que exploram a confiança entre navegador, usuário e aplicação (Singh & Chaudhary, 2022; Jang & Lee, 2025; Kshetri et al., 2024; Jovanović et al., 2006).

A evidência mostra que não existe solução universal. OAuth 2.0 oferece acesso delegado sem compartilhar senhas, JWT favorece arquiteturas escaláveis e sem sessão no servidor, e as defesas contra XSS/CSRF exigem camadas combinadas de validação, isolamento e verificação de origem (Patil, 2023; Pyroh et al., 2025; Fett et al., 2016; Weamie, 2022). Também há consenso de que flexibilidade excessiva, armazenamento inseguro de tokens e ausência de controles de navegador criam superfícies de ataque persistentes (Singh & Chaudhary, 2022; Vasylenko, 2025; Pratama et al., 2026; Gupta & Gupta, 2015).

Autenticação OAuth

OAuth 2.0 é um framework de autorização que permite a uma aplicação terceira obter **acesso limitado** a um serviço HTTP em nome do usuário ou em nome próprio, sem exigir o compartilhamento direto das credenciais do usuário (Hardt, 2012; Patil, 2023; Pyroh et al., 2025). Sua adoção é ampla em serviços web e nuvem, mas essa popularidade vem acompanhada de alta complexidade operacional e de múltiplas possibilidades de configuração, o que aumenta o risco de implementação insegura (Singh & Chaudhary, 2022; Kania & Górný, 2024; Patil, 2023).

Componentes e Funcionamento

A literatura descreve como elementos centrais do OAuth o resource owner, client, resource server, authorization server, access token e refresh token, cada um com função própria no controle de acesso (Patil, 2023). O token de acesso delimita escopos específicos de permissão, o que permite granularidade no acesso a recursos protegidos (Patil, 2023). Essa separação entre autenticação do usuário e delegação de acesso reduz a exposição de senhas, um ganho direto para privacidade e minimização de risco (Patil, 2023; Pyroh et al., 2025; Kania & Górný, 2024).

Principais Riscos

O ponto mais recorrente na literatura é que a **flexibilidade do OAuth 2.0** gera inconsistências entre implementações e favorece misconfigurations (Singh & Chaudhary, 2022; Patil, 2023; Kania & Górný, 2024). Uma análise formal encontrou **quatro ataques práticos** que comprometem a segurança do OAuth e também afetam OpenID Connect quando recomendações de segurança não são seguidas (Fett et al., 2016). Entre os vetores mais importantes estão redirecionamentos inseguros, uso incorreto do parâmetro state, inclusão de JavaScript de terceiros não confiável e presença de XSS ou open redirectors nos sites participantes (Fett et al., 2016).

Boas Práticas

A evidência aponta algumas salvaguardas consistentes. Todo o tráfego deve ocorrer sobre **SSL/TLS**, os redirection URLs devem ser pré-registrados e verificados, e os escopos devem ser reduzidos ao mínimo necessário (Patil, 2023). O parâmetro state precisa funcionar como nonce vinculado à sessão do usuário, com novo valor a cada tentativa de login, para bloquear CSRF no endpoint de redirecionamento (Fett et al., 2016). Há ainda propostas de endurecimento arquitetural, como convergir fluxos e tipos de clientes para reduzir complexidade, adicionar 2FA para clientes terceiros e reforçar o uso de assinaturas digitais e criptografia no ciclo dos tokens (Singh & Chaudhary, 2022).

JWT

JWT tornou-se um padrão dominante de **autenticação stateless** em aplicações web modernas e arquiteturas de microserviços porque encapsula informações assinadas em formato compacto e fácil de transportar (Pratama et al., 2026; Xu et al., 2024; Pyroh et al., 2025). Seu apelo técnico está em dispensar armazenamento de sessão no servidor e simplificar integração com APIs, plataformas móveis e ecossistemas distribuídos (Pyroh et al., 2025).

Vantagens Estruturais

Em sistemas HTTP, que são naturalmente sem conexão persistente, o cliente precisa demonstrar repetidamente que está autenticado, e tokens resolvem esse problema de forma eficiente (Jang & Lee, 2025). A literatura destaca que JWT é amplamente usado por sua simplicidade, velocidade e portabilidade em cenários como login social e aplicações distribuídas (Jang & Lee, 2025; Pyroh et al., 2025). Por isso, vários autores o tratam como solução escalável e conveniente para autenticação e autorização modernas, embora não necessariamente segura por padrão (Pratama et al., 2026; Pyroh et al., 2025; Zulkarneev & Basalay, 2024).

Vulnerabilidades Relevantes

A evidência recente é forte ao mostrar que implementações JWT introduzem falhas críticas. Um estudo em 43 bibliotecas, cobrindo 10 linguagens, encontrou **31 vulnerabilidades inéditas**, das quais 20 receberam CVEs, incluindo bypass de verificação de assinatura, spoofing e negação de serviço (Yang et al., 2026). Outro trabalho, ao analisar 358 aplicações open source, encontrou pelo menos **uma vulnerabilidade criptográfica em 65,92%** delas, indicando baixa maturidade no gerenciamento de chaves e uso de criptografia em JWT (Xu et al., 2024). Uma análise de 139 CVEs também observou crescimento relativo de falhas classificadas como **High e Critical** ao longo do ciclo de vida do JWT (Zulkarneev & Basalay, 2024).

Além disso, JWT não protege automaticamente a confidencialidade dos dados no payload. Se o token for roubado, informações podem ser expostas por simples decodificação, já que o formato é legível sem a chave privada quando não há cifragem adicional (Jang & Lee, 2025). A literatura também destaca riscos de roubo, reutilização, brute force, ataque “alg none”, algorithm confusion, injeção no parâmetro kid e ausência de revogação efetiva no servidor (Pratama et al., 2026; Bhuiya et al., 2026; Pyroh et al., 2025).

Medidas de Proteção

As medidas mais consistentes incluem **validação estrita de algoritmo**, rotação dinâmica de chaves, identificadores de chave seguros, armazenamento cifrado de chaves privadas e logging de auditoria (Pratama et al., 2026). O uso de JWKS com rotação dinâmica mostrou defesa robusta contra todos os vetores testados em um estudo de pen-test, com sobrecarga pequena, inferior a 50 ms nas consultas ao endpoint de chaves (Pratama et al., 2026). Outros trabalhos propõem cifrar partes do token para reduzir exposição de dados e combinar nonce, agilidade criptográfica, AES-GCM e detecção de anomalias para fortalecer integridade, confidencialidade e revogação (Jang & Lee, 2025; Bhuiya et al., 2026). No lado do cliente, armazenar JWT em **cookies HTTP-only** reduz acesso por scripts e, portanto, mitiga parte do risco de XSS sobre tokens (Dedhia, 2026; Vasylenko, 2025).

Segurança Contra XSS e CSRF

XSS e CSRF seguem entre as ameaças mais persistentes da web porque exploram relações de confiança já estabelecidas: no XSS, a aplicação entrega ao navegador um script malicioso; no CSRF, o site confia que uma requisição autenticada representa a intenção do usuário (Kshetri et al., 2024; Jovanović et al., 2006). As consequências mais citadas incluem roubo de cookies, sequestro de sessão, ações não autorizadas, comprometimento de dados e até instalação de malware (Kshetri et al., 2024; Manne, 2022; Gupta & Gupta, 2015; Weamie, 2022).

XSS

A literatura caracteriza o XSS como uma das vulnerabilidades mais prevalentes e graves das aplicações web modernas (Gupta & Gupta, 2015; Weamie, 2022). Uma revisão relata incidência aproximada em **70% das aplicações web**, associando o problema a falhas de desenvolvimento e validação incorreta de entrada (Weamie, 2022). O mecanismo central é a sanitização inadequada da entrada do usuário, que permite a injeção e execução de scripts no navegador da vítima (Kshetri et al., 2024; Weamie, 2022).

As defesas mais recorrentes são filtragem e escaping de entrada, técnicas server-side e client-side, abordagens por proxy e combinações de análise estática, dinâmica e auditoria de código (Kshetri et al., 2024; Weamie, 2022). Ainda assim, a evidência mostra limitação importante: a maior parte das soluções protege melhor contra XSS refletido e armazenado do que contra **DOM-based** e mutation-based XSS (Weamie, 2022).

CSRF

CSRF permite que o atacante faça o navegador da vítima enviar requisições arbitrárias a um site no qual ela já está autenticada (Jovanović et al., 2006; Kshetri et al., 2024; Barth et al., 2008). A literatura clássica também descreve o **login CSRF**, em que a vítima é autenticada no site legítimo como o próprio atacante, o que pode ter gravidade comparável à de uma falha XSS (Barth et al., 2008).

O mecanismo defensivo mais consolidado é o uso de **tokens anti-CSRF** aleatórios inseridos no formulário e validados no envio (Kshetri et al., 2024). A proteção se apoia no fato de que o atacante não consegue ler a resposta da página por causa da same-origin policy, o que dificulta recuperar o token válido (Kshetri et al., 2024). Há, porém, limitações práticas em tokens por requisição, como dificuldades com múltiplas janelas e outros contextos de envio; por isso, alguns trabalhos sugerem ampliar o escopo do token para a sessão ou empregar o modelo cookie-to-header token (Kshetri et al., 2024).

Outras defesas têm utilidade mais contextual. O cabeçalho Referer sofre bloqueios frequentes por razões de privacidade e, por isso, não é uma defesa universal; em HTTPS, porém, ele tende a ser confiável, especialmente contra login CSRF (Barth et al., 2008). Para o longo prazo, a proposta de uso do cabeçalho Origin busca preservar benefícios de segurança reduzindo problemas de privacidade (Barth et al., 2008). Há ainda abordagens automáticas por proxy no servidor e regras específicas em WAF para reduzir exploração em aplicações existentes (Jovanović et al., 2006; Kshetri et al., 2024).

Itens e Subitens do Tema

1. **Privacidade e controle de acesso:** dependem da separação entre autenticação, autorização e escopo mínimo de privilégios (Vasylenko, 2025; Patil, 2023).
 - 1.1. OAuth 2.0 reduz compartilhamento de credenciais, mas exige implementação rigorosa (Hardt, 2012; Patil, 2023; Fett et al., 2016).
 - 1.2. MFA, assinaturas digitais e criptografia complementam a proteção do ecossistema OAuth (Singh & Chaudhary, 2022; Vasylenko, 2025).
2. **Autenticação stateless com JWT:** favorece escalabilidade, integração com APIs e portabilidade entre serviços (Pratama et al., 2026; Pyroh et al., 2025).
 - 2.1. O principal risco está no gerenciamento de chaves, na validação de assinatura e no armazenamento do token (Xu et al., 2024; Pratama et al., 2026; Vasylenko, 2025).
 - 2.2. Rotação de chaves, whitelist de algoritmos, criptografia adicional e cookies HTTP-only endurecem a implementação (Pratama et al., 2026; Dedhia, 2026; Jang & Lee, 2025).
3. **Defesa contra XSS e CSRF:** requer múltiplas camadas, não uma única técnica (Weamie, 2022; Barth et al., 2008; Manne, 2022).
 - 3.1. Contra XSS, filtragem, escaping, revisão de código e técnicas híbridas seguem centrais, mas DOM-based XSS continua difícil de bloquear (Kshetri et al., 2024; Weamie, 2022).
 - 3.2. Contra CSRF, tokens sincronizados, cookie-to-header, state em OAuth e verificação de origem são os controles mais consistentes (Kshetri et al., 2024; Fett et al., 2016; Barth et al., 2008).

Conclusão

A pesquisa indica que privacidade e segurança em aplicações web dependem de uma arquitetura em camadas, na qual **OAuth 2.0**, **JWT** e proteções contra **XSS/CSRF** são complementares, não substitutos. OAuth 2.0 continua central para delegação segura de acesso, JWT é eficiente e escalável mas vulnerável a erros criptográficos e de armazenamento, e XSS/CSRF permanecem ameaças estruturais que exigem validação rigorosa, proteção de tokens e controles de origem. Em síntese, os melhores resultados aparecem quando protocolos modernos são combinados com MFA, criptografia, rotação de chaves, cookies HTTP-only, escopo mínimo e mitigação ativa de falhas do navegador e da aplicação.

Referências

As referências utilizadas nesta pesquisa correspondem aos 20 estudos fornecidos, com destaque para trabalhos normativos e analíticos sobre OAuth 2.0, JWT e XSS/CSRF, incluindo RFCs, análises formais, estudos de vulnerabilidades, surveys e avaliações experimentais (Hardt, 2012; Fett et al., 2016; Yang et al., 2026; Xu et al., 2024; Barth et al., 2008; Gupta & Gupta, 2015).

These search results were found and analyzed using Consensus, an AI-powered search engine for research. Try it at <https://consensus.app>. © 2026 Consensus NLP, Inc. Personal, non-commercial use only; redistribution requires copyright holders' consent.

References

- Barth, A., Jackson, C., & Mitchell, J. C. (2008). Robust defenses for cross-site request forgery. *Proceedings of the 15th ACM conference on Computer and communications security*. <https://doi.org/10.1145/1455770.1455782>
- Bhuiya, S., Shakil, G. M., Bashir, G. M. M., & Samsuzzaman, M. (2026). Ejwt: Enhancing JWT Security through Dynamic Cryptographic & LSTM Networks. *2026 5th International Conference on Electrical, Computer & Telecommunication Engineering (ICECTE)*, 1-6. <https://doi.org/10.1109/icecte69292.2026.11429432>
- Dedhia, N. (2026). Secure Web Authentication Using JWT and Role-Based Access Control. *INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*. <https://doi.org/10.55041/ijrem60274>
- Fett, D., Küsters, R., & Schmitz, G. (2016). A Comprehensive Formal Security Analysis of OAuth 2.0. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. <https://doi.org/10.1145/2976749.2978385>
- Gupta, S., & Gupta, B. (2015). Cross-Site Scripting (XSS) attacks and defense mechanisms: classification and state-of-the-art. *International Journal of System Assurance Engineering and Management*, 8, 512 - 530. <https://doi.org/10.1007/s13198-015-0376-0>
- Hardt, D. (2012). The OAuth 2.0 Authorization Framework. *RFC*, 6749, 1-76. <https://doi.org/10.17487/rfc6749>
- Jang, S.-W., & Lee, S.-H. (2025). Vulnerabilities and Encryption Applications of JWT-Based Authentication Methods. *Journal of Information Systems Engineering and Management*. <https://doi.org/10.52783/jisem.v10i8s.1055>
- Jovanović, N., Kirda, E., & Krügel, C. (2006). Preventing Cross Site Request Forgery Attacks. *2006 Securecomm and Workshops*, 1-10. <https://doi.org/10.1109/seccomw.2006.359531>
- Kania, D., & Górny, P. (2024). Assessing the security status of systems using the OAuth 2.0 authorization protocol. *Computer Science and Mathematical Modelling*. <https://doi.org/10.5604/01.3001.0055.0853>
- Kshetri, N., Kumar, D., Hutson, J., Kaur, N., & Osama, O. (2024). AlgoXSSF: Detection and Analysis of Cross-Site Request Forgery (XSRF) and Cross-Site Scripting (XSS) Attacks via Machine Learning Algorithms. *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, 1-8. <https://doi.org/10.1109/isdfs60797.2024.10527278>
- Manne, T. A. K. (2022). Mitigating Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) in Java Web Applications. *Journal of Mathematical & Computer Applications*. [https://doi.org/10.47363/jmca/2022\(1\)214](https://doi.org/10.47363/jmca/2022(1)214)
- Patil, K. (2023). Authentication and Authorization in Web Applications. *Journal of Engineering and Applied Sciences Technology*. [https://doi.org/10.47363/jeast/2023\(5\)186](https://doi.org/10.47363/jeast/2023(5)186)
- Pratama, F. A., Hermanto, A., & Kusnanto, G. (2026). Security Assessment of JWKS-Based Authentication: Mitigating JWT Attack Vectors Through Penetration Testing. *Jurnal Teknik Informatika (Jutif)*. <https://doi.org/10.52436/1.jutif.2026.7.2.5662>
- Pyroh, M., Tereshchuk, G., & Toroshanko, O. (2025). AUTHENTICATION PRINCIPLES AS SECURITY ASPECTS OF WEB DEVELOPMENT. *MEASURING AND COMPUTING DEVICES IN TECHNOLOGICAL PROCESSES*. <https://doi.org/10.31891/2219-9365-2025-81-36>
- Singh, J., & Chaudhary, N. (2022). OAuth 2.0 : Architectural design augmentation for mitigation of common security vulnerabilities. *J. Inf. Secur. Appl.*, 65, 103091. <https://doi.org/10.1016/j.jisa.2021.103091>

Vasylenko, O. (2025). COMPARATIVE ANALYSIS OF MODERN AUTHENTICATION AND AUTHORIZATION PROTOCOLS FOR WEB APPLICATIONS. *Terra security*. <https://doi.org/10.20998/3083-6298.2025.02.01>

Weamie, S. J. Y. (2022). Cross-Site Scripting Attacks and Defensive Techniques: A Comprehensive Survey. *International Journal of Communications, Network and System Sciences*.
<https://doi.org/10.4236/ijcns.2022.158010>

Xu, B., Jia, S., Lin, J., Zheng, F., Y., Liu, L., Gu, X., & Song, L. (2024). JWTKey: Automatic Cryptographic Vulnerability Detection in JWT Applications. 263-282. https://doi.org/10.1007/978-3-031-51479-1_14

Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W., & Wang, B. (2026). Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discovery. *Proceedings 2026 Network and Distributed System Security Symposium*. <https://doi.org/10.14722/ndss.2026.240697>

Zulkarneev, I., & Basalay, K. A. (2024). JSON Web Tokens Lifecycle-Based Threat Classification. *2024 IEEE 25th International Conference of Young Professionals in Electron Devices and Materials (EDM)*, 1920-1924.
<https://doi.org/10.1109/edm61683.2024.10615042>